

Física Computacional

ano letivo: 2021/2022

docente: Nuno Castro [nuno.castro@fisica.uminho.pt]

Licenciatura em Física e Licenciatura / Mestrado Integrado em Engenharia Física

9ª aula (09/12/2021)



Séries de Fourier

• Uma função (periódica, com período T) $f(t)$ pode ser expressa como uma combinação linear de funções sinusoidais:

❖ Função periódica

$$f(t + T) = f(t) \quad T = 2\pi$$

❖ limites reais

$$a_n = \frac{1}{\pi} \int_0^{2\pi} f(t) \cos(nt) dt$$

❖ pode ser complexa ou real

$$b_n = \frac{1}{\pi} \int_0^{2\pi} f(t) \sin(nt) dt$$

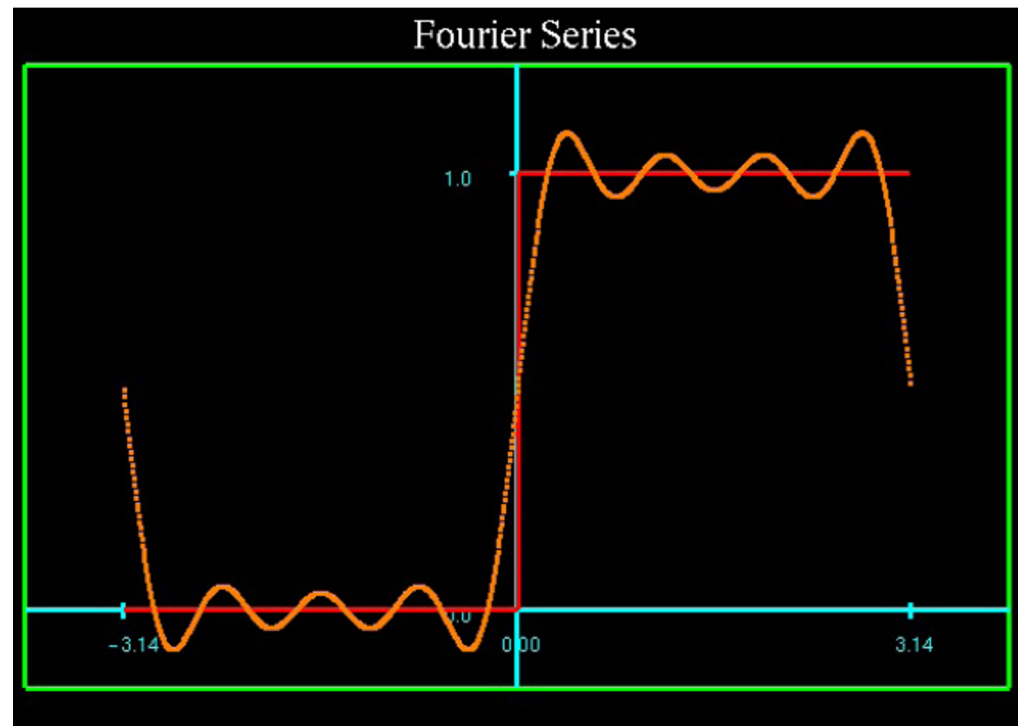
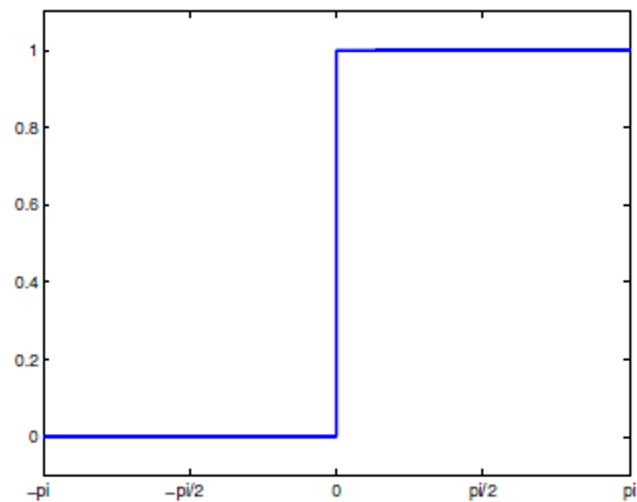
$$c_n = \frac{1}{2\pi} \int_0^{2\pi} f(t) \exp(-jnt) dt \quad n = 0, \pm 1, \pm 2, \dots$$

$$c_n = \frac{1}{T} \int_{-T/2}^{T/2} f(t) \exp(-j \frac{n2\pi}{T} t) dt \quad n = 0, \pm 1, \pm 2, \dots$$



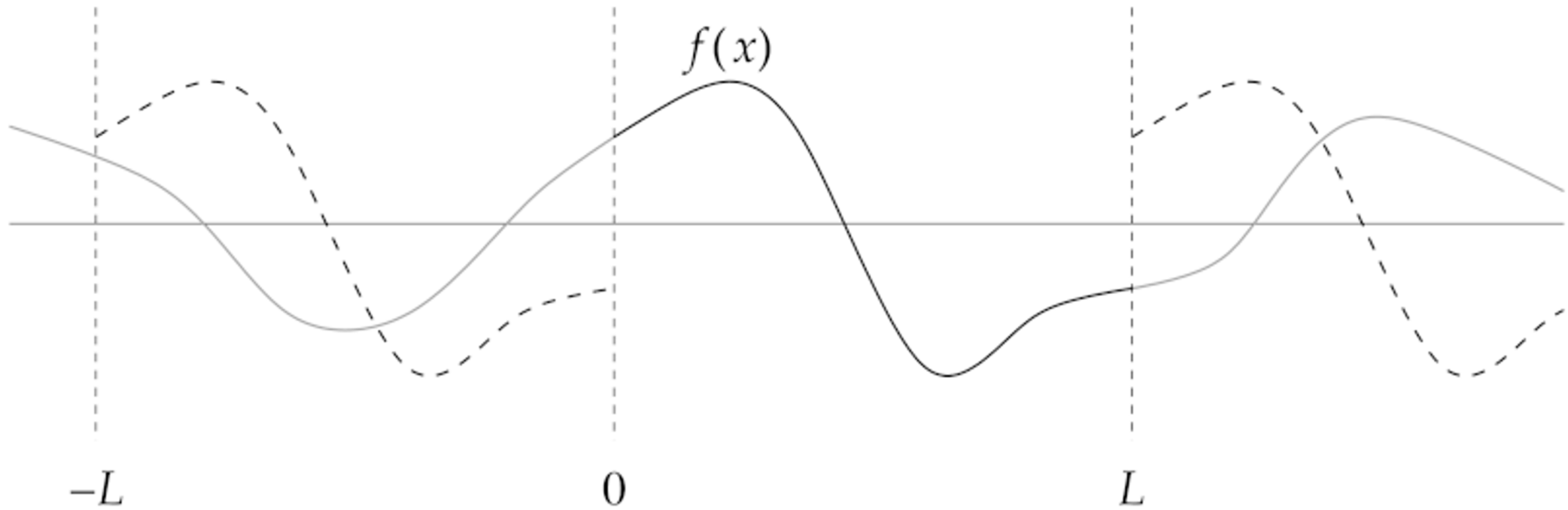
$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

Séries de Fourier



Séries de Fourier

.O que fazer com funções não periódicas?



Transformadas de Fourier

- Função aperiódica - T torna-se infinito
- da Série chegamos à Transformada

$$\frac{2\pi}{T} \rightarrow \omega$$


$$\begin{aligned} f(t) &= \frac{1}{T} \sum_{k=-\infty}^{\infty} \exp(jk\omega t) \left[\int_{-T/2}^{T/2} f(t) \exp\left(-j\frac{n2\pi}{T}t\right) dt \right] \\ &= \frac{1}{2\pi} \sum_{k=-\infty}^{\infty} \omega \int_{-\infty}^{\infty} f(\xi) \exp(jk\omega(t - \xi)) d\xi \\ &= \frac{1}{2\pi} \int_{-\infty}^{\infty} \exp(j\omega t) d\omega \int_{-\infty}^{\infty} f(\xi) \exp(-j\omega\xi) d\xi \\ &= \frac{1}{2\pi} \int_{-\infty}^{\infty} F(\omega) \exp(j\omega t) d\omega \end{aligned}$$

Transformadas de Fourier

- Na maioria dos casos só é possível obter uma aproximação numérica
- No mundo digital - tempo discreto e frequência discreta

$$\Delta t = \frac{T}{N} \quad N = \# \text{ de amostras}$$

$$t_k = k\Delta t \quad k = 0, 1, \dots, N - 1$$

$$f_k = f(t_k)$$

$$\omega_n = 2\pi \frac{n}{T} \quad \omega_0 = \frac{2\pi}{T}$$

Transformadas de Fourier

Forward Discrete Fourier Transform (DFT):

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i 2\pi k n / N}$$

Inverse Discrete Fourier Transform (IDFT):

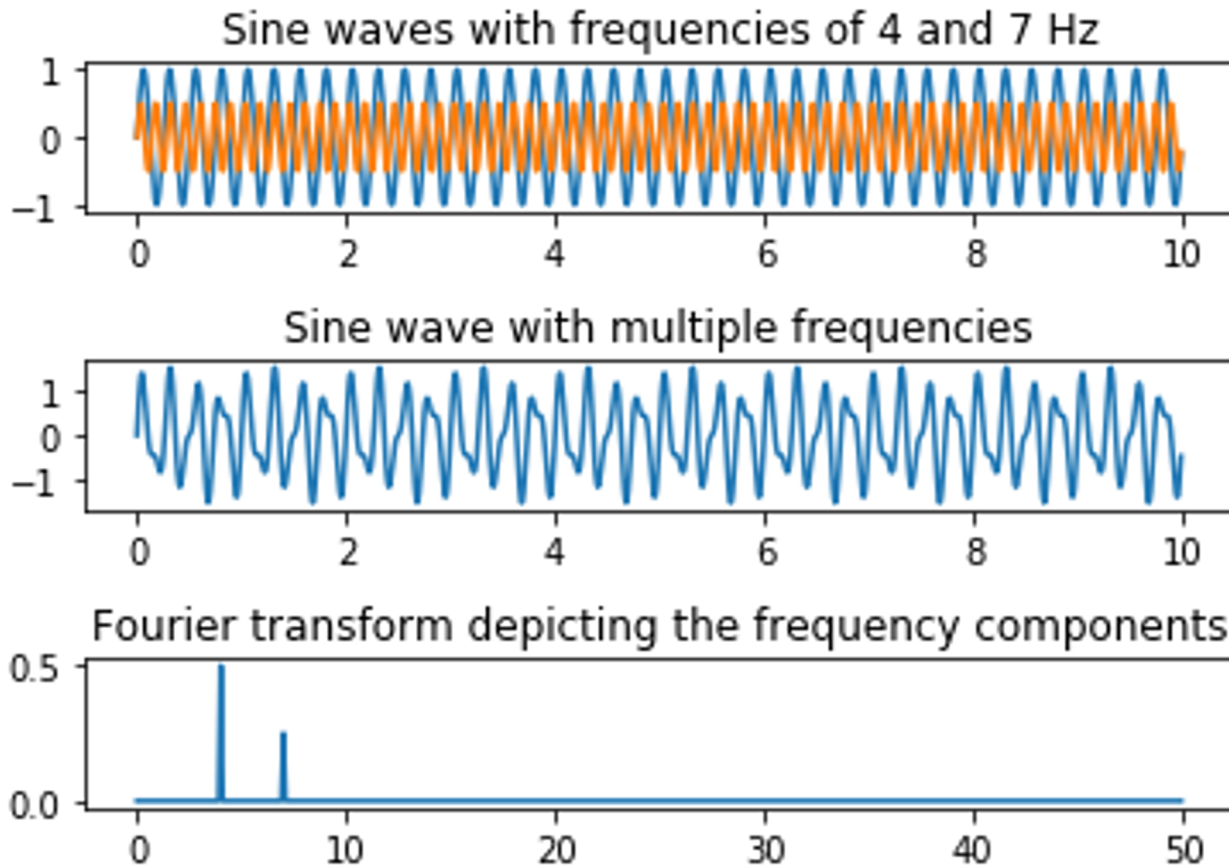
$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i 2\pi k n / N}$$

Transformadas de Fourier

```
In [1]: 1 import numpy as np
        2 def DFT_slow(x):
        3     """Compute the discrete Fourier Transform of the 1D array x"""
        4     x = np.asarray(x, dtype=float)
        5     N = x.shape[0]
        6     n = np.arange(N)
        7     k = n.reshape((N, 1))
        8     M = np.exp(-2j * np.pi * k * n / N)
        9     return np.dot(M, x)
        10
        11 x = np.random.random(1024)
        12 np.allclose(DFT_slow(x), np.fft.fft(x))
```

Out[1]: True

Transformadas de Fourier



Transformadas de Fourier

```
1 %timeit DFT_slow(x)
2 %timeit np.fft.fft(x)
```

213 ms $\pm$ 8.48 ms per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

44.8 μ s $\pm$ 1.09 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10000 loops each)

Transformadas de Fourier

•Fast Fourier Transforms (FFT): explorando a simetria do problema

$$\begin{aligned} X_k &= \sum_{n=0}^{N-1} x_n \cdot e^{-i 2\pi k n / N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i 2\pi k (2m) / N} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i 2\pi k (2m+1) / N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i 2\pi k m / (N/2)} + e^{-i 2\pi k / N} \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i 2\pi k m / (N/2)} \end{aligned}$$

Transformadas de Fourier

```
def FFT(x):  
    """A recursive implementation of the 1D Cooley-Tukey FFT"""  
    x = np.asarray(x, dtype=float)  
    N = x.shape[0]  
  
    if N % 2 > 0:  
        raise ValueError("size of x must be a power of 2")  
    elif N <= 32: # this cutoff should be optimized  
        return DFT_slow(x)  
    else:  
        X_even = FFT(x[::2])  
        X_odd = FFT(x[1::2])  
        factor = np.exp(-2j * np.pi * np.arange(N) / N)  
        return np.concatenate([X_even + factor[:N / 2] * X_odd,  
                                X_even + factor[N / 2:] * X_odd])
```

Transformadas de Fourier

```
%timeit DFT_slow(x)  
%timeit FFT(x)  
%timeit np.fft.fft(x)
```

10 loops, best of 3: 77.6 ms per loop

100 loops, best of 3: 4.07 ms per loop

10000 loops, best of 3: 24.7 μ s per loop

Transformadas de Fourier

```
def FFT_vectorized(x):  
    """A vectorized, non-recursive version of the Cooley-Tukey FFT"""  
    x = np.asarray(x, dtype=float)  
    N = x.shape[0]  
  
    if np.log2(N) % 1 > 0:  
        raise ValueError("size of x must be a power of 2")  
  
    # N_min here is equivalent to the stopping condition above,  
    # and should be a power of 2  
    N_min = min(N, 32)  
  
    # Perform an O[N^2] DFT on all length-N_min sub-problems at once  
    n = np.arange(N_min)  
    k = n[:, None]  
    M = np.exp(-2j * np.pi * n * k / N_min)  
    X = np.dot(M, x.reshape((N_min, -1)))  
  
    # build-up each level of the recursive calculation all at once  
    while X.shape[0] < N:  
        X_even = X[:, :X.shape[1] / 2]  
        X_odd = X[:, X.shape[1] / 2:]  
        factor = np.exp(-1j * np.pi * np.arange(X.shape[0])  
                        / X.shape[0])[ :, None]  
        X = np.vstack([X_even + factor * X_odd,  
                       X_even - factor * X_odd])  
  
    return X.ravel()
```

Transformadas de Fourier

```
x = np.random.random(1024 * 16)
%timeit FFT(x)
%timeit FFT_vectorized(x)
%timeit np.fft.fft(x)
```

10 loops, best of 3: 72.8 ms per loop
100 loops, best of 3: 4.11 ms per loop
1000 loops, best of 3: 505 μ s per loop

$$\frac{dv(t)}{dt} - v(t) = e^t$$

$$\frac{d^2x(t)}{dt^2} - 5\frac{dx(t)}{dt} + 2x(t) = \cos(t)$$

x(t): função desconhecida

t: variável independente

Uma EDO é linear se:

- - $x(t)$ e as suas derivadas tiverem potência 1
- - não houver produtos entre $x(t)$ e as suas derivadas (ou entre elas)

$$\frac{dx(t)}{dt} - x(t) = e^t$$

EDO linear

$$\frac{d^2x(t)}{dt^2} - 5\frac{dx(t)}{dt} + 2t^2x(t) = \cos(t)$$

EDO linear

$$\left(\frac{d^2x(t)}{dt^2}\right)^3 - \frac{dx(t)}{dt} + \sqrt{x(t)} = 1$$

EDO não linear

para obter soluções únicas para uma EDO de ordem n são necessárias n condições

$$\frac{d^2 x(t)}{dt^2} + 4x(t) = 0$$

EDO de 2ª ordem

$$x(0) = a$$

$$\dot{x}(0) = b$$

precisamos de 2 condições



problemas de condição inicial

as condições auxiliares são para o mesmo ponto da variável independente

$$\ddot{x} + 2\dot{x} + x = e^{-2t}$$

$$x(0) = 1, \dot{x}(0) = 2.5$$

igual

problemas de condição fronteira

as condições auxiliares não para pontos diferentes da variável independente

resolução mais difícil

$$\ddot{x} + 2\dot{x} + x = e^{-2t}$$

$$x(0) = 1, x(2) = 1.5$$

diferente

Exemplo: amortecedor para uma máquina industrial

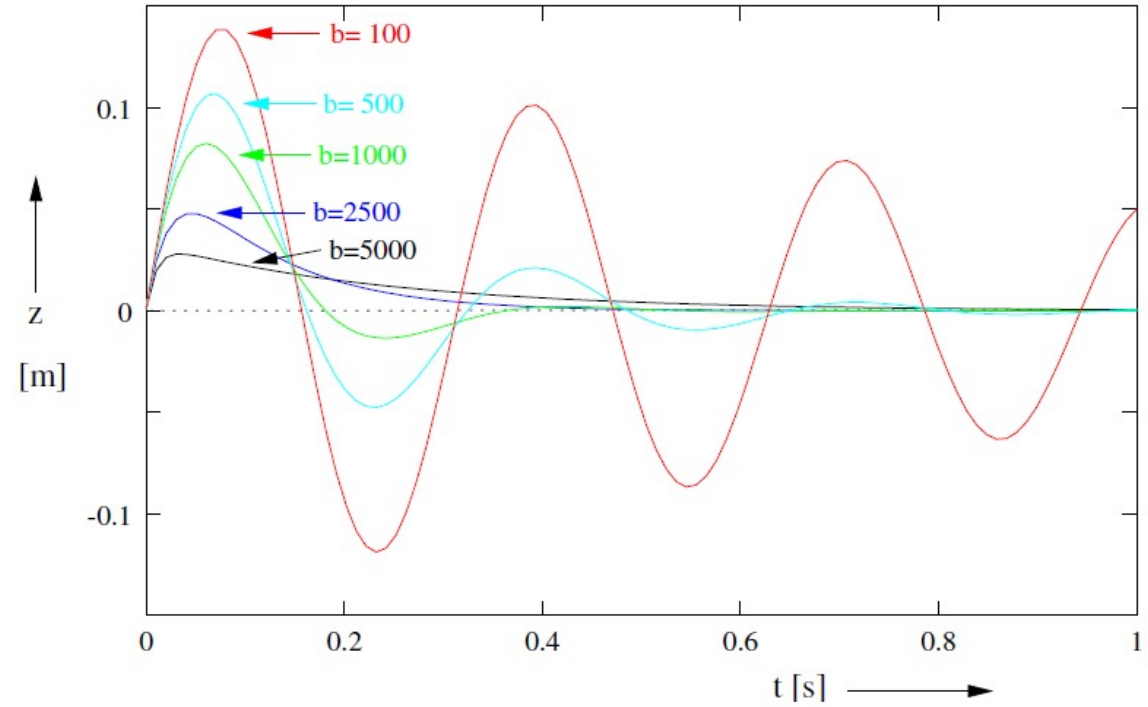
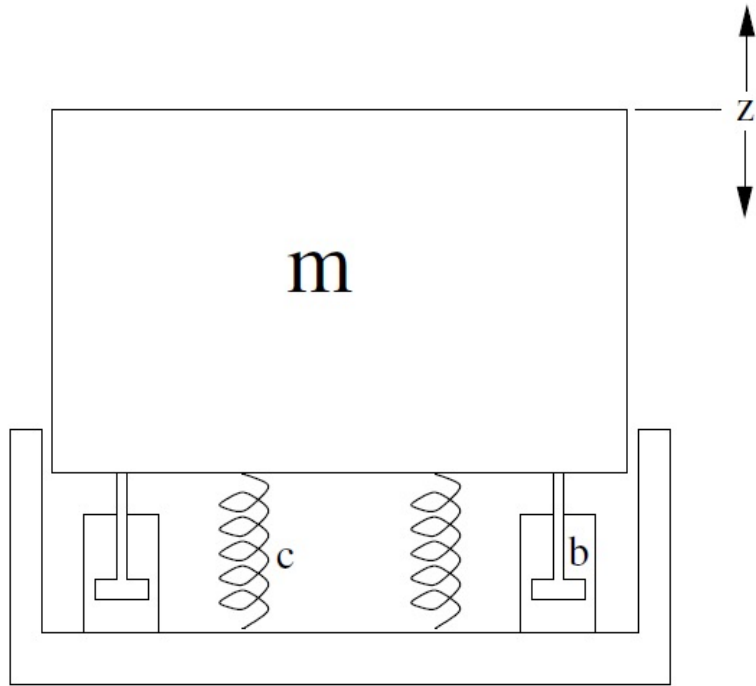
- massa da máquina: $m = 50.0 \text{ kg}$
- constante de elasticidade: $c = 2.0 \cdot 10^4 \frac{N}{m}$
- constante de amortecimento $b = 2500 \frac{kg}{s}$

A oscilação do aparelho é descrito pela **equação diferencial**

$$m \frac{d^2 z}{dt^2} + b \frac{dz}{dt} + cz = 0$$

No instante inicial $t = 0$ a máquina está

- na posição $z = 0.00100 \text{ m}$
- com uma velocidade $\frac{dz}{dt} = 3,00 \frac{m}{s}$



Este exemplo ainda tem uma solução analítica.

Mas **a equação do movimento é muito simplificada.**

Na realidade,

- a força de uma mola não é exactamente linear;
- a força de um amortecedor não depende linearmente da velocidade.

Nestes casos são necessários **métodos numéricos** para determinar a solução da equação de movimento.

As equações de primeira ordem, da forma geral:

$$y' = f(x, y)$$

ou as de ordem mais elevada, da forma

$$y^{(n)} = f(x, y, y', y'', \dots, y^{(n-1)})$$

têm solução da forma

$$y = \Phi(x)$$

- Os **métodos analíticos**, sempre que disponíveis, permitem encontrar explicitamente a relação, e desse modo y fica conhecido para todos os valores de x .
- Por outro lado, os **métodos numéricos** dão-nos valores de y correspondentes apenas a um conjunto finito de valores de x , a solução é obtida como uma **tabela de valores**, em vez de através duma função contínua.
- Além disso, a **solução analítica** — caso exista — é **exacta**, enquanto que uma **solução numérica** está afectada de erro que embora pequeno, pode desvirtuar a solução exacta se não for controlado.

Portanto teremos que ter em conta dois aspectos ao resolver numericamente equações diferenciais:

- ① o método em si e
- ② a sua exactidão ou precisão.

Uma maneira usual de tratar uma equação de 2ª ordem ou de ordem superior é substituí-la por um sistema equivalente de equações da 1ª ordem. Isto pode fazer-se pela introdução duma **variável intermédia** $z = y'$, a qual permite, que a equação

$$y'' = f(x, y, y')$$

seja substituída pelo sistema

$$\begin{aligned} y' &= z \\ x' &= f(x, y, z) \end{aligned}$$

Com uma equação de ordem mais elevada podemos também introduzir novas variáveis para $y'', y''', \dots, y^{(n-1)}$. Por esta razão concentrar-nos-emos totalmente em **métodos para a resolução de equações de 1^a ordem**.

Existindo uma relação $y = \Phi(x)$, entre x e y , a relação pode escrever-se

$$y' = f(x, \Phi(x)) =: g(x)$$

Não conhecemos $\Phi(x)$ e portanto $g(x)$ também não. Contudo a solução pode escrever-se formalmente como

$$y = \int g(x)dx + A$$

onde A é uma constante de integração. Para determinar A basta conhecer uma **condição inicial**; por exemplo, $y = y_0$ para $x = x_0$. Um sistema com uma condição adicional assim chama-se **problema de valor inicial**. A solução vem então

$$y = y_0 + \int_{x_0}^x g(t)dt$$

Note-se que qualquer problema de valor inicial pode ser convertido numa **equação integral** deste tipo. Com efeito, se $y = \phi(x)$ é solução do problema $\frac{dy}{dx} = f(x, y)$ com $y(x_0) = y_0$, tem-se

$$y - y_0 = \phi(x) - \phi(x_0) = \int_{x_0}^x f(t, \phi(t)) dt$$

$$\Rightarrow y = y_0 + \int_{x_0}^x f(t, \phi(t)) dt$$